

Module 10 – Application Debugging (React)

Module 10 – Application Debugging Overview: This module focuses on debugging React applications using widely adopted tools such as Chrome DevTools and Visual Studio Code. Learners will gain practical experience in identifying, analyzing, and resolving frontend issues by inspecting the DOM, debugging TypeScript code, and examining application state. The module emphasizes efficient debugging practices to improve code quality and application performance.

29 Learning Objectives:

- After completing this module, learners will be able to:
- Debug React applications using Chrome DevTools, including DOM inspection and breakpoints
- Analyze JavaScript and TypeScript code using the Sources panel and source maps
- Inspect elements and understand application structure through the DOM tree
- Configure and use Visual Studio Code debugger with React applications
- Set breakpoints, watches, and inspect variables in TypeScript files
- Apply effective debugging techniques to identify and fix frontend issues
- Improve application reliability and performance through systematic debugging

What is Application Debugging?

Application debugging is the process of finding, analyzing, and fixing errors (bugs) in a software application. In React applications, debugging helps developers identify issues related to components, state management, props, API calls, routing, rendering, and user interactions.

Debugging is an essential skill because even well-written code can contain logical mistakes, runtime errors, or unexpected behavior.

Why is Debugging Important?

Debugging helps developers:

- Identify coding errors quickly.
- Improve application performance.
- Reduce development time.
- Ensure a better user experience.
- Understand how the application works internally.
- Detect memory leaks and unnecessary component rendering.
- Fix JavaScript and TypeScript errors efficiently.

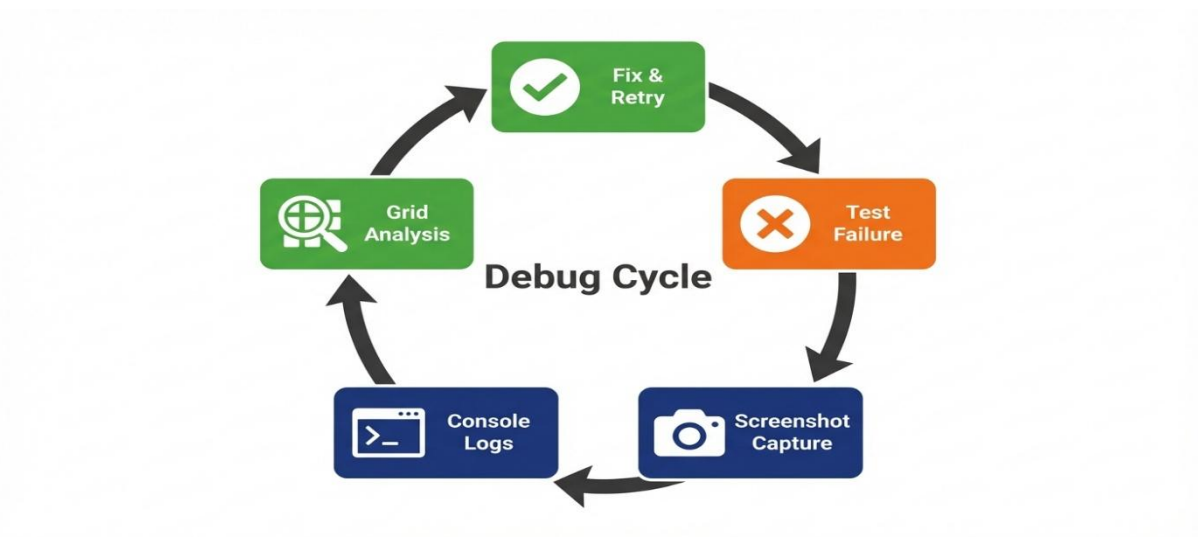
Types of Errors in React Applications

Error Type	Description	Example
Syntax Error	Mistakes in code structure	Missing } or)
Runtime Error	Error while executing code	Undefined variable
Logical Error	Code runs but gives wrong output	Incorrect calculation

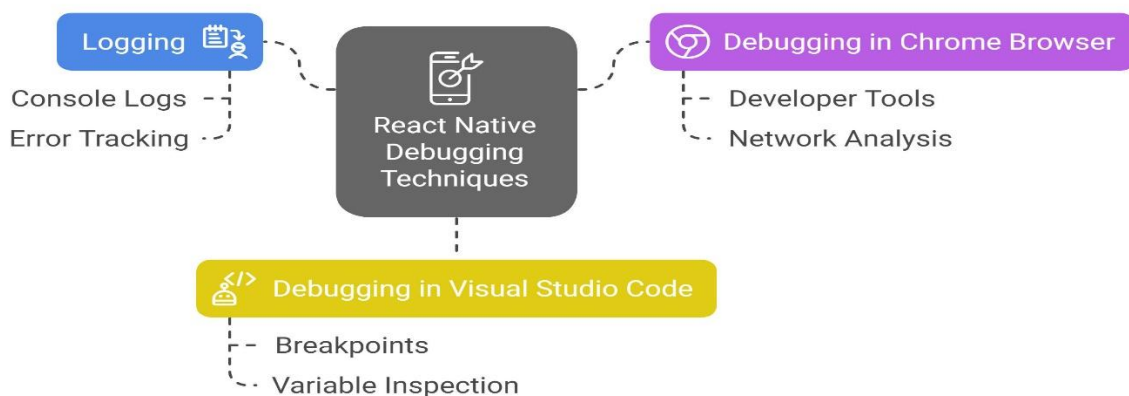
Error Type	Description	Example
API Error	Backend request failure	404 or 500 response
Rendering Error	Component fails to display	Invalid JSX
State Error	Incorrect state update	Infinite re-render

React Application Debugging Workflow

The Debug Cycle is a structured approach used to detect, analyze, correct, and verify software defects. Instead of fixing errors randomly, developers follow a sequence of steps that helps them identify the root cause of the problem, implement a solution, and test the application until the issue is completely resolved.



React Native Debugging Techniques



Step 1: Test Failure

A debugging process begins when a test or application behaves unexpectedly.

Examples:

- Login button does not work.
- API request returns an error.
- React component does not render.
- Incorrect output is displayed.
- Application crashes.

Purpose

- Identify that a problem exists.
- Observe the symptoms of the bug.
- Determine when and where the error occurs.

Step 2: Screenshot Capture

After identifying the issue, developers capture screenshots of the error.

A screenshot helps by:

- Recording the exact error.
- Sharing the issue with team members.
- Comparing the application's behavior before and after the fix.
- Documenting bugs for future reference.

Examples:

- Browser error page
- Broken user interface
- Console error message
- Incorrect layout

Step 3: Console Logs

The browser console provides detailed information about application execution.

Developers use:

`console.log()`

`console.warn()`

Why Console Logs are Important

- Check variable values.
- Verify API responses.
- Track execution flow.
- Detect runtime errors.

Chrome DevTools Console is commonly used for this purpose.

Step 4: Grid Analysis

This step represents analyzing all available debugging information to determine the **root cause** of the problem.

Developers inspect:

- Application logic
- Source code
- Component hierarchy
- State and props
- Network requests
- DOM structure
- Variable values

Step 5: Fix & Retry

Once the root cause is identified, the developer modifies the code to resolve the issue.

Example: Incorrect code

```
let total = price - tax;
```

Correct code

```
let total = price + tax;
```

After fixing the issue:

- Save the code.
- Restart or refresh the application if necessary.
- Run the application again.
- Execute the same test.

Repeat the Cycle

If the issue still exists:

Fix-Test Again-Failure-Analyze-Fix Again

The cycle repeats until the application behaves as expected.

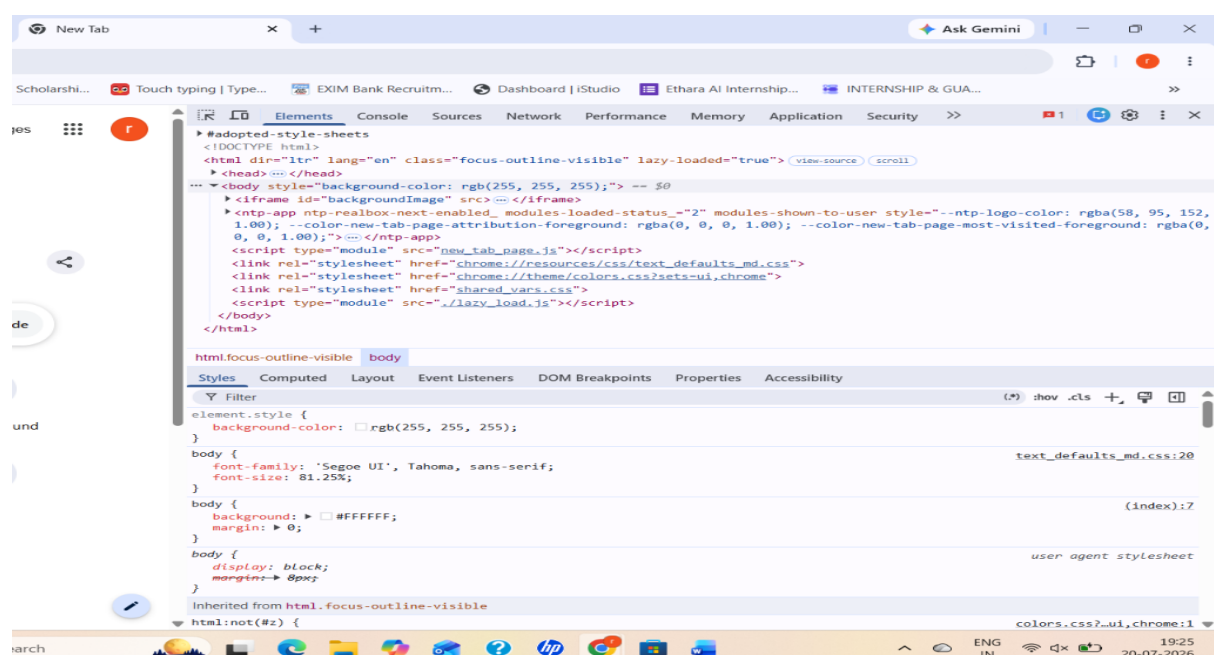
Tool	Purpose
Chrome DevTools	Inspect HTML, CSS, DOM, Console, Network and Performance
React Developer Tools	Inspect React components, props, state and hooks
Visual Studio Code Debugger	Set breakpoints, inspect variables and step through code
Browser Console	Display logs, warnings and runtime errors
Network Tab	Analyze API requests and responses

Chrome DevTools

Chrome DevTools is a browser-based debugging and development tool that enables developers to inspect webpage elements, monitor application behavior, analyze network activity, debug JavaScript and TypeScript code, and optimize the performance of web applications.

- HTML
- CSS
- JavaScript
- React Components
- Network Requests
- Performance
- Memory Usage

Opening Chrome DevTools



Method 1

Press

F12

Method 2

Ctrl + Shift + I

Method 3

Right Click

Inspect

Chrome DevTools Overview

1. Elements Tab

The Elements tab is used to inspect and edit the HTML and CSS of a webpage. It displays the Document Object Model (DOM) in a tree structure, allowing developers to understand how elements are organized.

Uses

- Inspect HTML elements
- View and modify CSS styles
- Analyze the DOM structure
- Check applied classes and IDs
- Test layout changes without modifying source code

Example HTML

```
<div class="container">  
  <h1>Welcome to React</h1>  
  <button>Click Me</button>  
</div>
```

In the Elements panel, this structure appears as a DOM tree where each HTML tag is represented as a node.

Benefits

- Helps identify missing or incorrectly placed elements.
- Allows temporary editing of HTML and CSS for testing.
- Useful for debugging layout and styling issues.

2. Console Tab

The Console tab displays JavaScript messages, errors, warnings, and output generated using `console.log()`.

Uses

- Display log messages
- View JavaScript errors
- Display warnings
- Test JavaScript expressions
- Debug runtime issues

Example

```
console.log("React Started");
```

Output

React Started

The Console also displays runtime errors.

Benefits

- Helps verify variable values.
- Detects runtime exceptions.

3. Sources Tab

The Sources tab is one of the most powerful debugging tools in Chrome DevTools. It allows developers to debug JavaScript and TypeScript code by pausing execution and inspecting application behavior.

Features

- Pause code execution
- Add and remove breakpoints
- Step through code line by line
- Inspect variable values
- Debug original TypeScript code using source maps

Example

Suppose the following function contains a bug:

```
function calculateTotal(price, tax) {
```

```
    let total = price + tax;  
    return total;  
}
```

A breakpoint can be placed on the line:

```
let total = price + tax;
```

When execution reaches this line, it pauses, allowing developers to inspect:

```
price = 100
```

```
tax = 18
```

```
total = ?
```

Benefits

- Simplifies bug detection.
- Allows inspection of variable values during execution.
- Helps identify logical and runtime errors.

4. Network Tab

The Network tab monitors all network requests made by the application, such as API calls, images, stylesheets, and JavaScript files.

Information Displayed

- API requests
- Response time
- Request headers
- Response headers
- HTTP status codes
- File sizes

Example

```
GET /api/users
```

```
Status: 200 OK
```

```
Time: 120 ms
```

If an API fails:

```
Status: 404 Not Found
```

or

Status: 500 Internal Server Error

Benefits

- Helps identify API failures.
- Measures request performance.
- Verifies server responses.
- Useful for debugging backend communication.

5. Performance Tab

The Performance tab analyzes how efficiently a webpage runs. It records browser activity and provides detailed insights into rendering and execution performance.

Measures

- Rendering speed
- CPU usage
- Frame rate (FPS)
- JavaScript execution time
- User interactions

Uses

Developers can record application activity while interacting with the webpage to identify performance bottlenecks.

Benefits

- Detects slow rendering.
- Identifies expensive JavaScript operations.
- Helps optimize React component performance.

6. Memory Tab

The Memory tab is used to analyze memory usage and identify memory-related problems in web applications.

Used For

- Detecting memory leaks
- Garbage collection analysis
- Heap snapshots

- Memory allocation tracking

Inspecting the DOM Tree

The Document Object Model (DOM) is a programming interface for HTML and XML documents. When a web page loads, the browser converts the HTML code into a tree-like structure called the DOM Tree.

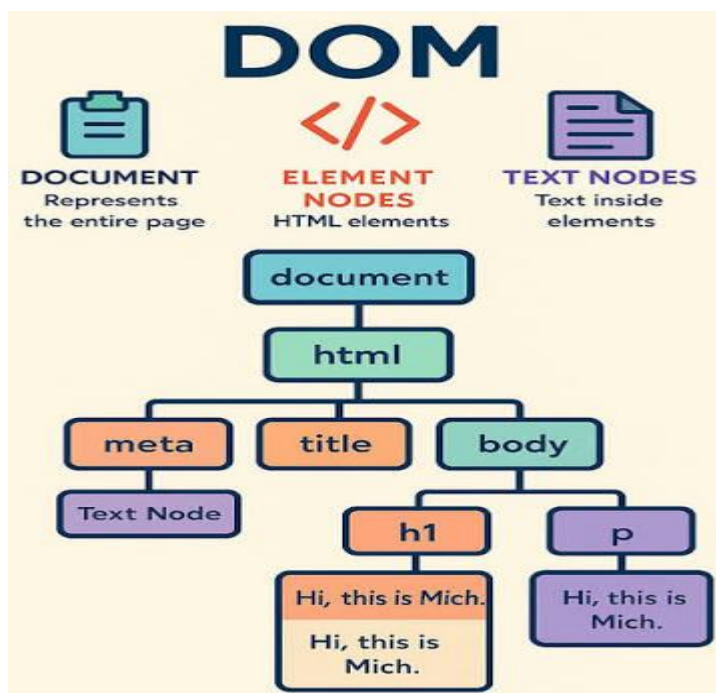
Each HTML element becomes an object (called a node), allowing JavaScript and React to access, modify, add, or remove elements dynamically without reloading the page.

DOM Tree

Body

```

└─ Div
    └─ H1
        └─ Button
  
```



1. Document Node

- Represents the entire HTML page.
- It is the root object created by the browser.
- Every webpage starts with the document object.

Example:

```
<!DOCTYPE html>
```

<html>

In JavaScript

document.title

The browser accesses the webpage through the document object.

2. Element Nodes

These are the HTML tags present in a webpage.

Examples:

- <html>
- <body>
- <div>
- <h1>
- <p>
- <button>

Each HTML tag becomes an Element Node inside the DOM tree.

Example

<body>

<h1>Welcome</h1>

<p>Hello Students</p>

</body>

3. Text Nodes

The text written inside an HTML element becomes a Text Node.

Example

<h1>Hello React</h1>

Here,

- h1 - Element Node
- Hello React -Text Node

Explanation

- Document is the root node.
- html is the parent element.

- meta, title, and body are child elements of html.
- body contains two child elements:
 - h1
 - p
- The text inside h1 and p becomes Text Nodes.

Parent and Child Relationship

The DOM follows a hierarchical tree structure.

- body is the parent node.
- h1 and p are child nodes.
- The text inside them is a text node.

Benefits:

- Locate missing elements.
- Check CSS classes.
- Verify nested components.
- Edit HTML live for testing.

Breakpoints

A breakpoint pauses program execution.

A Breakpoint is a debugging marker placed on a specific line of code that pauses program execution when that line is reached, allowing developers to inspect the application's current state and identify errors.

Why are Breakpoints Important?

Breakpoints are used to:

- Pause program execution at a specific line.
- Inspect variable values.
- Analyze the flow of execution.
- Detect logical and runtime errors.
- Verify state and props in React components.
- Debug JavaScript and TypeScript code efficiently.
- Reduce the need for excessive console.log() statements.

Types of Breakpoints

Breakpoint	Purpose
Line Breakpoint	Pause at specific line
Conditional Breakpoint	Pause only if condition is true
DOM Breakpoint	Pause when DOM changes
Event Breakpoint	Pause on click, keyboard, mouse events
XHR Breakpoint	Pause during API request

Types of Breakpoints

1. Line Breakpoint

A Line Breakpoint pauses execution whenever a specific line of code is reached.

Example

```
let total = price + tax;
```

Use when:

- Checking variable values.
- Verifying calculations.
- Finding logical errors.

2. Conditional Breakpoint

A Conditional Breakpoint pauses execution only when a specified condition is true.

Benefits

- Avoids unnecessary pauses.
- Useful inside loops.
- Saves debugging time.

3. DOM Breakpoint

A DOM Breakpoint pauses execution whenever the structure or attributes of a selected HTML element change.

Uses

- Debug dynamic UI updates.
- Detect unexpected DOM modifications.
- Analyze rendering issues.

4. Event Listener Breakpoint

An Event Listener Breakpoint pauses execution when a specified browser event occurs.

Examples of events:

- Mouse Click
- Keyboard Press
- Mouse Hover
- Scroll
- Touch Events

Example

```
<button onClick={saveData}>
```

Execution pauses immediately when the button is clicked.

Uses

- Debug click events.
- Debug form submissions.
- Analyze user interactions.

5. XHR / Fetch Breakpoint

XHR Means XMLHttpRequest This breakpoint pauses execution whenever an API request is sent.

Example

```
fetch("/api/users");
```

Execution pauses before the request is made.

Uses

- Debug REST API calls.
- Verify request parameters.
- Inspect server responses.

Source Maps

React applications are bundled and often minified, making debugging difficult. Source maps map the generated JavaScript back to the original source files (including TypeScript), allowing you to debug the code you actually wrote instead of compiled output.

Benefits:

- Debug original React components.
- Set breakpoints in .tsx or .ts files.
- Read meaningful line numbers and variable names.

React Developer Tools

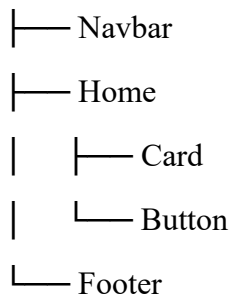
React Developer Tools is a browser extension designed specifically for React.

Features:

- Inspect component hierarchy.
- View props.
- Monitor state.
- Check hooks.
- Profile rendering performance.

Example component tree:

App



Debugging TypeScript in React

TypeScript adds static typing, helping catch many errors during development.

Example

```
let age:number = 20;
```

```
console.log(age);
```

If you write:

```
let age:number = "Twenty";
```

TypeScript immediately reports a type error.

Visual Studio Code Debugger

VS Code includes a built-in debugger for React applications.

Features:

- Breakpoints
- Variable inspection
- Call stack
- Watch expressions
- Debug Console
- Step execution

Configuring VS Code Debugger

1. Open the React project.
2. Click the Run and Debug icon.
3. Select Create a launch.json if prompted.
4. Choose Chrome or Microsoft Edge.
5. Start the React application.
6. Press F5 to begin debugging.

Watches

A watch lets you monitor a variable or expression during execution.

Call Stack

The call stack shows the sequence of function calls leading to the current execution point.

Example

App()

↓

Login()

↓

validateUser()

↓

checkPassword()

This helps trace where an error originated.

Variable Inspection

During debugging, you can inspect:

- Local variables
- Global variables
- Props
- State
- Function parameters

Example

```
username = "Rithika"
```

```
age = 20
```

```
loggedIn = true
```

Step Execution

Command	Purpose
Continue	Resume execution
Step Over	Execute current line without entering functions
Step Into	Enter a function call
Step Out	Exit the current function
Restart	Restart the debugging session
Stop	End debugging

Common React Debugging Techniques

- Read error messages carefully in the Console.
- Use `console.log()` to inspect values.
- Check whether props are passed correctly.

- Verify state updates.
- Inspect API responses in the Network tab.
- Use React Developer Tools to inspect components and hooks.
- Set breakpoints to observe execution flow.
- Watch important variables for unexpected changes.
- Use source maps to debug original TypeScript files.

Advantages of Application Debugging

- Faster error detection.
- Improved code quality.
- Better application performance.
- Easier maintenance.
- Increased software reliability.

Application debugging is a crucial part of React development. By using tools such as Chrome DevTools, React Developer Tools, and the Visual Studio Code Debugger, developers can inspect the DOM, analyze JavaScript and TypeScript execution, monitor component state and props, debug API calls, and systematically identify and fix issues. Mastering these debugging techniques leads to more reliable, maintainable, and high-performing React applications.